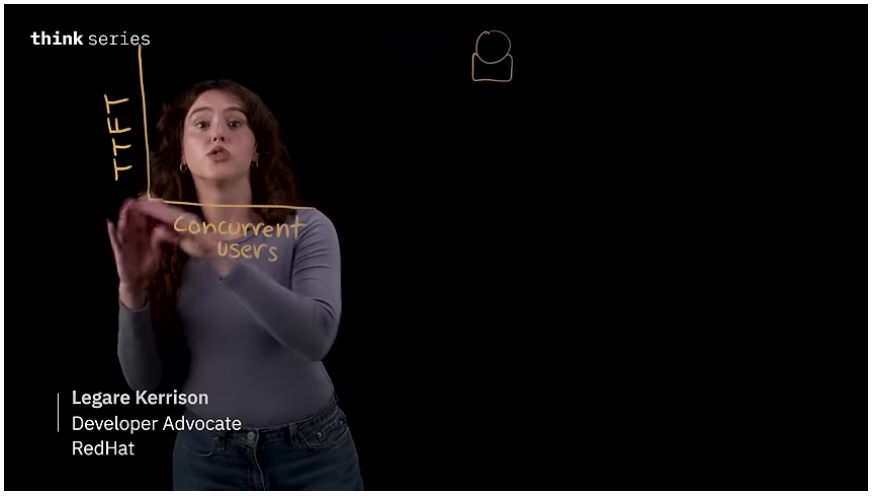
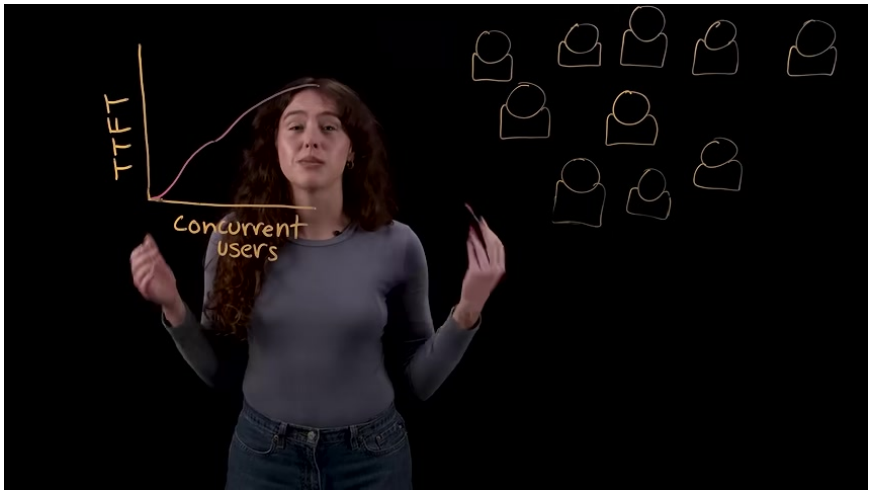
Inference Scaling, KV Cache, and Deployment Tuning

Why inference slows down at scale

When a single user is querying a model, the time to first token is usually very fast. As concurrency rises, though, latency increases, GPU memory pressure grows, and throughput drops. The core problem is often not the model itself, but how inference handles memory as context accumulates token by token.

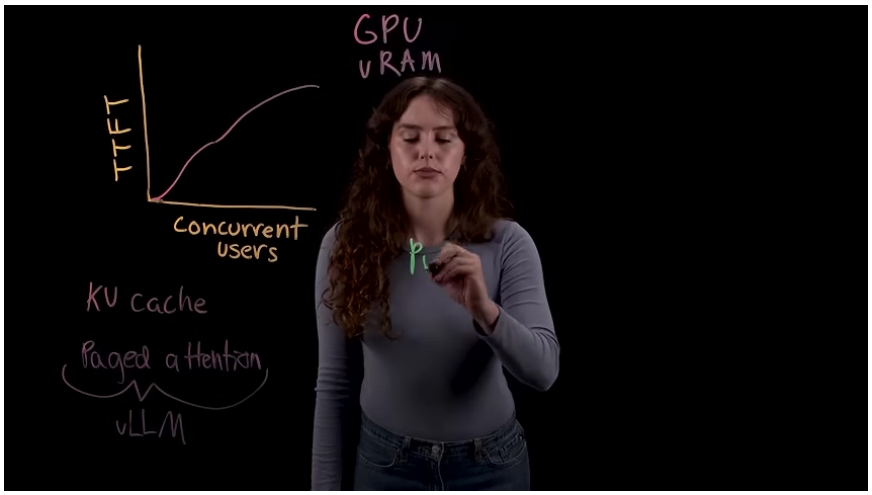


A presenter stands beside a hand-drawn graph labeled TTF vs concurrent users.

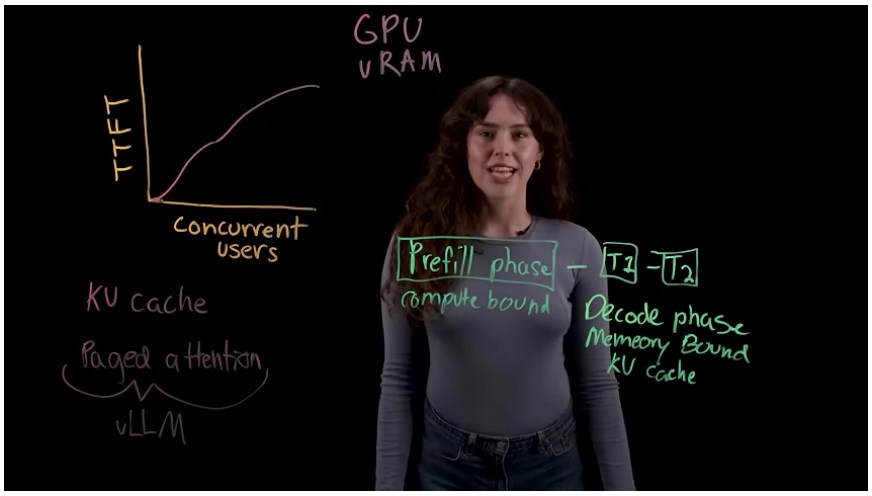


The presenter gestures in front of a completed graph showing TTF increasing with concurrent users.

The explanation begins with the two main stages of inference. In the pre-fill stage, the model processes the prompt and builds an internal representation of it, which is compute-heavy and responsible for the initial delay before output appears. In the decode stage, the model generates new tokens one at a time and repeatedly reaches back into GPU memory to retrieve the growing context, making this phase memory-bound.



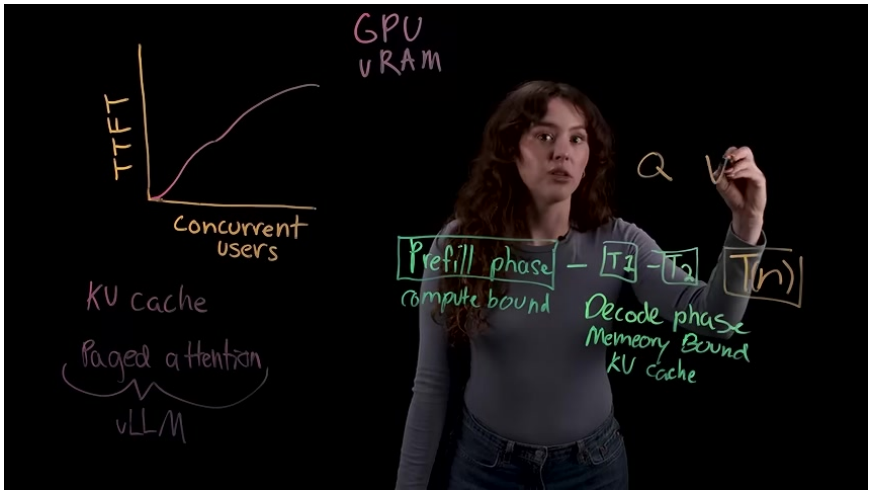
A presenter writes the word 'prefill' on the transparent board in front of the same GPU memory and KV cache diagram.



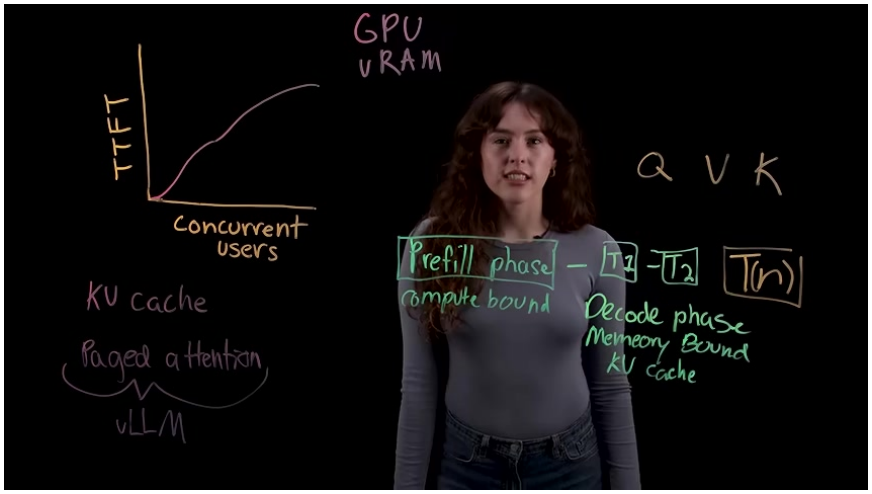
A presenter stands beside a diagram labeling the prefill phase as compute-bound and the decode phase as memory-bound.

KV cache and paged attention

During decoding, each transformer layer produces query, key, and value vectors. Without a cache, each new token would have to recompute the keys and values for all prior tokens, which becomes extremely expensive for long outputs. KV cache avoids that repeated work by storing those key and value matrices so the next token can attend to the existing history instead of rebuilding it from scratch.

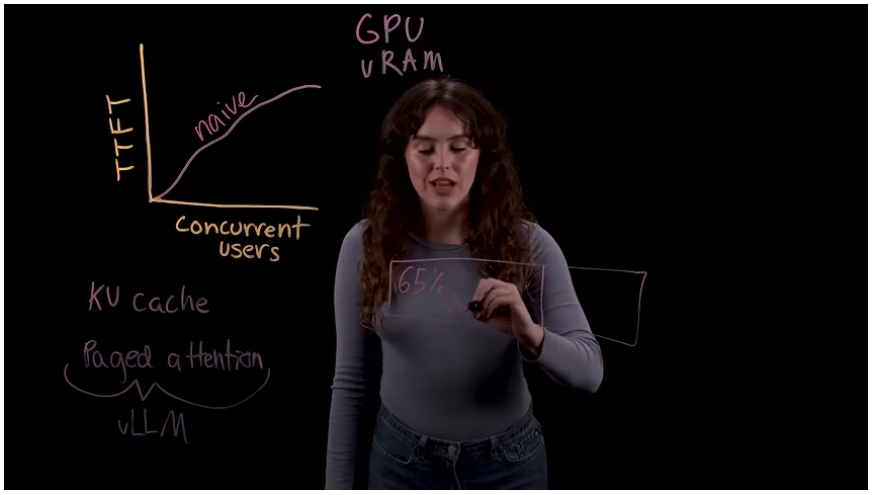


The presenter writes additional labels near the diagram, including Q, V, and K, while explaining relevance in the context of KV cache.



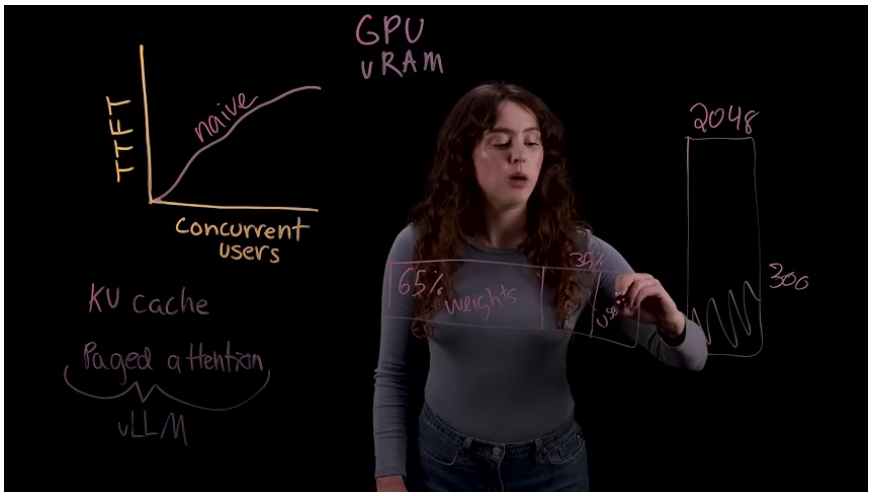
The diagram remains on screen with the presenter centered and the Q, V, K labels visible to the right.

The catch is that memory usage can become inefficient very quickly in a traditional serving setup. A large model can already consume most of the available VRAM just for weights, leaving a limited remainder for KV cache. If each request is assigned a large contiguous reservation based on maximum sequence length, much of that allocation goes unused, creating internal fragmentation, external fragmentation, and redundant duplication of shared prompts.



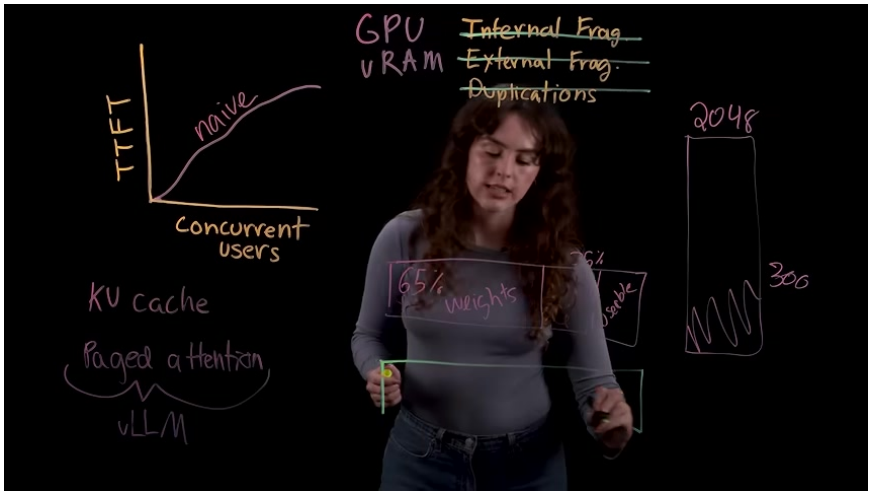
The presenter writes a transparent box labeled 65% over the hand-drawn graph about TTFT, concurrent users, GPU VRAM, and paged attention.

The presenter stands beside a diagram showing 2048 and a boxed region labeled 65% weights.

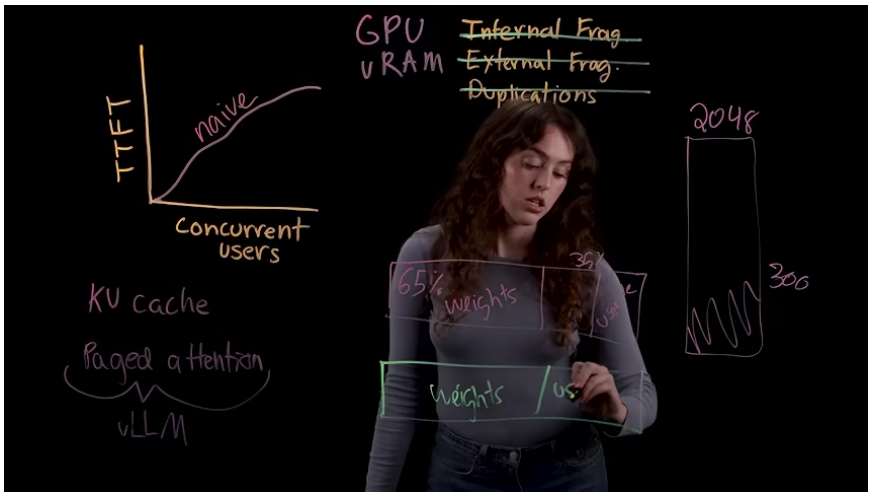


The presenter explains a memory and throughput diagram showing TTFT versus concurrent users, GPU VRAM, KV cache, paged attention, and a 65% weights / 35% usable split.

Paged attention addresses this by managing KV cache more like an operating system manages RAM. Instead of one large contiguous block per request, it splits cache into small fixed-size pages that can live anywhere in GPU memory and be mapped logically to physically scattered locations. This makes memory allocation far more flexible and reduces wasted space.



A presenter stands before a transparent board covered with handwritten notes and sketches about GPU VRAM, KV cache, and concurrency.



The presenter continues explaining the board, with highlighted boxes for weights and reserved GPU memory.

Deployment tuning for better throughput and latency

The video closes with several practical deployment knobs for getting more out of the same hardware. First is GPU memory utilization, which determines how much of the remaining VRAM is allowed for KV cache. Higher values can increase concurrency on stable workloads, while lower values can help avoid out-of-memory failures during spikes.

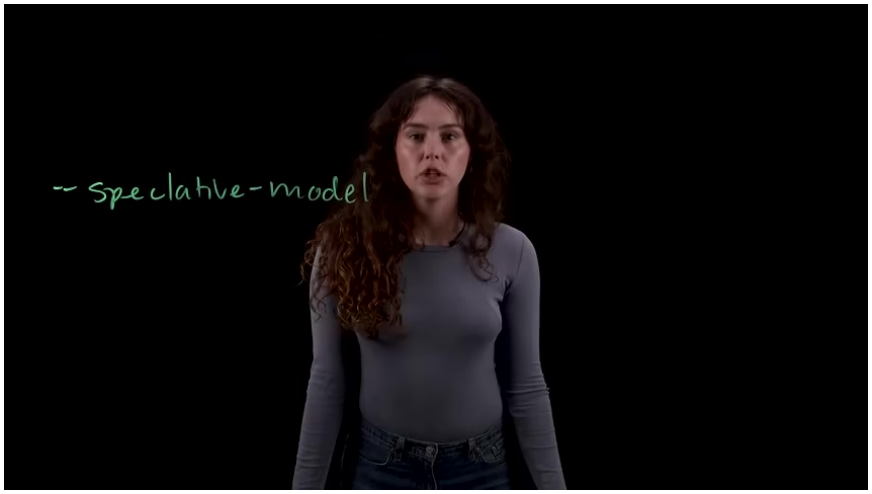
The presenter stands under the heading 'Tuning vLLM Deployments' on a mostly blank transparent board.

The board now shows '01 Tune - gpu memory utilization' under the main deployment tuning title.

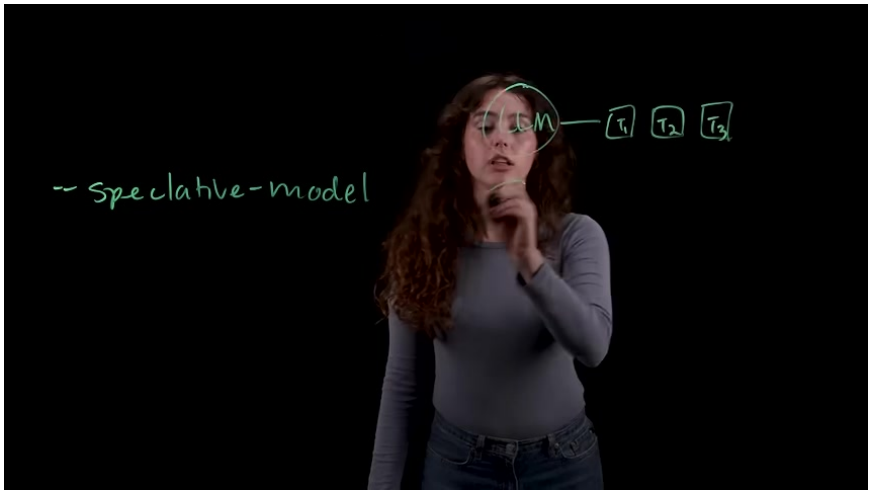
Second is prefix caching. Because paged attention can hash KV blocks by token sequence, requests that share the same system prompt can reuse the same stored context rather than recomputing it. This is especially effective in chat, RAG, and agentic workflows, where the first prompt segment is often repeated across many requests.

A presenter stands in front of a black glass board titled "Tuning vLLM Deployments" with numbered tuning tips written in marker.

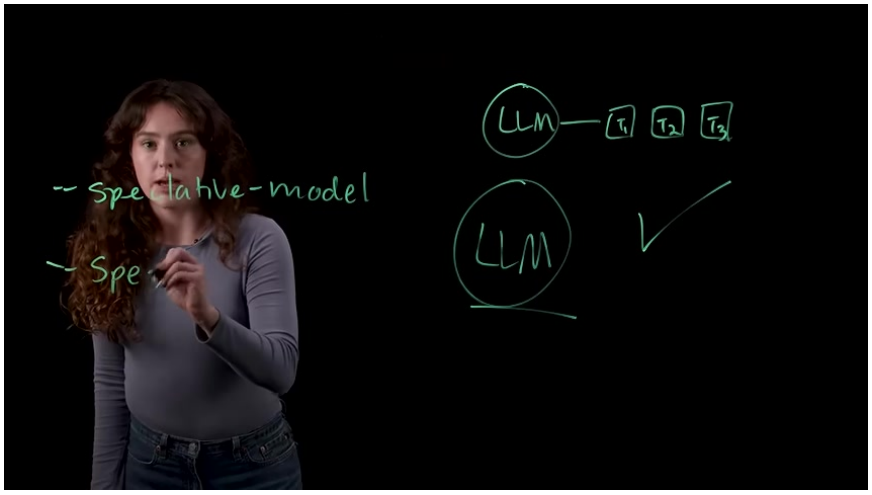
Third is chunked prefill, which changes scheduling so decode requests get served first and any leftover compute is used to process prompt chunks. That prevents long prompts from stalling streamed output and can substantially improve throughput in production. For latency-sensitive use cases, the speaker also recommends speculative decoding, where a small draft model proposes tokens and the larger model verifies them in batches. The output remains equivalent to running the large model alone, but interactive responsiveness improves when concurrency is not already saturating the GPU.



A presenter draws a speculative model diagram on a black transparent board.



A presenter adds labeled boxes T1, T2, and T3 to the LLM diagram.



A presenter writes a simple diagram showing an LLM feeding tokens and a check mark beside it.